42 COMMON CORE

# cub3D: Complete Code Workflow & Engine Architecture

A First-Person 3D Raycasting Engine in C (Wolfenstein 3D) — Evaluation Guide

Language & Standard
Pure C (C99)
42 Norminette Compliant

Graphics Engine
MiniLibX (X11 / Cocoa)
Direct Software Buffer

Core Algorithm
DDA Grid Raycaster
Perpendicular Projection

# 01. THE MENTAL MODEL: HOW 2D BECOMES 3D

The Raycasting Illusion: Transforming a 2D matrix into a first-person 3D viewport

Why Raycasting was Revolutionary (1992)

- The Map is strictly 2D: an array of integer cells where '1' = wall, '0' = walkable floor.
- The Player is a 2D coordinate (pos_x, pos_y) with a facing angle.
- Instead of complex 3D polygon rasterization, we cast ONE 2D ray per horizontal screen column (800 rays for an 800x600 window).
- When a ray hits a wall at distance D, we draw a vertical line on screen.
- Line height is inversely proportional: LineHeight = SCREEN_HEIGHT / PerpWallDist.
- Far walls look short; close walls look tall. Result: instant 3D depth perception!

2D World Map Grid vs 3D Projected Screen



# 02. OVERALL CODE WORKFLOW: 4 PHASES

Sequential execution architecture across modules and files

Phase 1: Parse & Validate
src/parse/

• check_file_extension(.cub)
• read_file() via GNL
• parse_element (NO,SO,WE,EA,F,C)
• parse_map & find_player
• validate_map_enclosed (BFS)

Phase 2: Graphics Setup
src/drawing/ & textures/

• mlx_init() connection
• mlx_new_window(800, 600)
• mlx_new_image (Framebuffer)
• mlx_get_data_addr (raw RAM)
• load_textures (4 XPM images)

Phase 3: Render Loop
src/raycast/ & controls/

• handle_keys (sliding movement)
• draw_floor_and_ceiling (RGB)
• raycast_walls (800 DDA rays)
• draw_minimap (bonus radar)
• mlx_put_image_to_window

Phase 4: Cleanup & Exit
src/utils/

• close_game (unified exit)
• free_2d_array (map)
• free_textures (4 paths & imgs)
• mlx_destroy_image/window
• Zero leaks under Valgrind!

# 03. PHASE 1: ENTRY POINT & FILE READING

Code Locations: src/main.c & src/parse/parse_utils.c

src/main.c: The Setup

```
int main(int argc, char **argv)
{
    t_game game;

    if (argc != 2)
        error_exit("Usage: ./cub3D map.cub");
    ft_memset(&game, 0, sizeof(t_game));
    game.floor_colour = -1;
    game.ceiling_colour = -1;
    parse(&game, argv[1]);
    init_render(&game);
    return (0);
}
```

- Zeroes all struct memory (prevents wild pointers).
- Sets floor and ceiling to -1 (so black 0x000000 can be distinguished from uninitialized).

src/parse/parse_utils.c: Safe Reading

1. Extension Validation:

```
void check_file_extension(char *path)
{
    int len = ft_strlen(path);
    if (len < 5 || ft_strncmp(path + len - 4, ".cub", 4) != 0)
        error_exit("File must have .cub extension");
}
```

2. Memory Array Read via GNL:

- count_file_lines(path): opens file, counts total lines, closes fd.
- malloc(sizeof(char *) * (count + 1)): allocates exact array.
- read_file(path): re-opens file, stores each line using get_next_line().
- Advantage: eliminates arbitrary static buffer limits!

# 04. PHASE 1: TEXTURES & RGB BITWISE PACKING

Code Location: src/parse/parse_element.c

Element Parsing & Duplicate Checking

```
void parse_element(t_game *game, char *line)
{
    if (ft_strncmp(line, "NO ", 3) == 0)
        set_if_empty("NO texture", 0, game, ft_strtrim(line + 3, "
\t\n"));
    else if (ft_strncmp(line, "SO ", 3) == 0)
        set_if_empty("SO texture", 1, game, ft_strtrim(line + 3, "
\t\n"));
    ...
    else if (ft_strncmp(line, "F ", 2) == 0)
        game->floor_colour = parse_rgb(line + 2);
}
```

• set_if_empty(): checks if path already exists; if yes -> error_exit("Duplicate...").
• Trims whitespace and tabs ('\t') defensively.

parse_rgb(): Bitwise Math & Zero Leaks

```
int parse_rgb(char *str)
{
    // 1. Verify exactly 2 commas (e.g. "220,100,0")
    // 2. ft_split(str, ',')
    // 3. Verify 3 tokens, each with 1-3 pure digits via
is_valid_num()
    // 4. Validate range: [0, 255]
    // 5. Pack into single 32-bit int:
    return (r << 16 | g << 8 | b);
}
```

Why (r << 16 | g << 8 | b)?
R: 220 (0xDC) << 16 = 0x00DC0000
G: 100 (0x64) << 8  = 0x00006400
B:   0 (0x00)       = 0x00000000
Combined with OR (|) = 0x00DC6400

# 05. PHASE 1: MAP GRID & PLAYER SPAWN SETUP

Code Locations: src/parse/parse_map.c & src/parse/player_setting.c

src/parse/parse_map.c: Map Integrity

• find_map_start(): locates first map row. Enforces that map is the LAST element (no textures after map).
• parse_map(): reads rows into game->map char**.
• Empty line detection: if an empty line occurs within the map body, raises error_exit("Invalid line or empty line inside map").
• find_map_cols(): measures maximum row width to allocate bounding geometry.

src/parse/player_setting.c: Spawn Logic

```
void find_player(t_game *game)
{
    // Scans for 'N', 'S', 'E', 'W'
    // Enforces found == 1 (no zero, no multiple spawns)
    game->player.pos_x = col + 0.5;
    game->player.pos_y = row + 0.5;
    set_player_direction(&game->player, dir);
    game->map[row][col] = '0'; // Consumed into walkable floor
}
```

Why col + 0.5 and row + 0.5?
• Grid coordinates point to the top-left corner of the cell.
• +0.5 places player in the exact center of the cell, equidistant from walls, preventing initial clipping glitches!

# 06. PHASE 1: MAP ENCLOSURE VIA ITERATIVE BFS

Code Locations: src/parse/map_validation.c & src/parse/space_handling.c

Why Queue BFS instead of Recursive Flood Fill?

• The Stack Problem: On large open maps (e.g. 100x100), recursive flood_fill() pushes 10,000+ stack frames, crashing the program with Stack Overflow.
• Our Solution: Iterative Queue (t_queue) on the heap:
  - FIFO: head_idx processes front, tail_idx enqueues neighbors.
  - Checks 4 cardinal neighbors (Up, Down, Left, Right).
  - Guaranteed O(Rows * Cols) execution with constant stack size!

Two-Pass Inner vs Outer Space Algorithm

External padding spaces are legal; internal holes are illegal!

Pass 1: mark_outer_spaces()
• BFS starting from all outer borders.
• Changes true exterior spaces into 'X'.

Pass 2: flood_fill_bfs() from Player
• If flood touches 'X' -> Leaked to exterior!
  error_exit("Map is not enclosed by walls");
• If flood touches ' ' -> Internal hole inside map!
  error_exit("Map contains hole (space inside map)");

Result: 100% precision on irregular/ragged maps!

# 07. PHASE 2: GRAPHICS SETUP & DOUBLE BUFFERING

Code Locations: src/drawing/mlx_tools.c & src/textures/texture.c

src/drawing/mlx_tools.c: Image Framebuffer

```
void init_render(t_game *game)
{
    game->mlx = mlx_init();
    game->win = mlx_new_window(game->mlx, 800, 600, "cub3D");
    game->img = mlx_new_image(game->mlx, 800, 600);
    game->addr = mlx_get_data_addr(game->img, &bpp, &line_len,
&endian);
    load_textures(game);
    mlx_loop_hook(game->mlx, render_frame, game);
    mlx_hook(game->win, 2, 1L<<0, key_press, game);
    mlx_hook(game->win, 3, 1L<<1, key_release, game);
    mlx_hook(game->win, 17, 0, close_game, game);
    mlx_loop(game->mlx);
}
```

Why Double Buffering via Raw Memory?

• The Mistake: Calling mlx_pixel_put() per pixel triggers 480,000 X11 window server calls per frame! Screen flickers and drops to 2 FPS.
• The Solution: my_mlx_pixel_put() writes directly to game->addr in RAM:

```
void my_mlx_pixel_put(t_game *game, int x, int y, int color)
{
    char *dst = game->addr + (y * line_len + x * (bpp / 8));
    *(unsigned int *)dst = color;
}
```

• Once all 800 columns are drawn in memory, mlx_put_image_to_window() pushes the finished buffer once. Zero tearing, rock-solid 60+ FPS!

# 08. PHASE 3: THE RENDER_FRAME() PIPELINE

What happens every single frame: 60 times per second

src/drawing/mlx_tools.c: render_frame()

```
int render_frame(t_game *game)
{
    // Step 1: Process User Controls
    handle_keys(game);

    // Step 2: Draw Background
    draw_floor_and_ceiling(game);

    // Step 3: 3D Raycasting Core
    raycast_walls(game);

    // Step 4: Bonus Radar Overlay (if compiled)
#ifdef BONUS
    draw_minimap(game);
#endif

    // Step 5: Hardware Screen Refresh
    mlx_put_image_to_window(game->mlx, game->win
```

Detailed Step Breakdown

1. handle_keys():
Reads keyboard state table game->keys[65536]. Updates player position with wall sliding and rotates camera vectors.

2. draw_floor_and_ceiling():
Loops y from 0 to 300 (top half) -> fills with ceiling_colour.
Loops y from 300 to 600 (bottom half) -> fills with floor_colour.

3. raycast_walls():
Loops column x = 0 to 799. Calculates ray direction, runs DDA, computes wall distance, and draws textured vertical stripe over background.

4. mlx_put_image_to_window():
Displays finished frame buffer.

# 09. RAYCASTING STEP 1: CAMERA VECTORS & RAYS

Code Location: src/raycast/raycast_main.c: init_ray_vars()

```
init_ray_vars(t_game *game, int x)

// 1. Map screen x [0, 800) to camera space [-1.0, +1.0]
game->ray.camera_x = 2.0 * x / (double)SCREEN_WIDTH - 1.0;

// 2. Compute Ray Direction Vector
game->ray.ray_dir_x = player.dir_x + player.plane_x * camera_x;
game->ray.ray_dir_y = player.dir_y + player.plane_y * camera_x;

// 3. Grid Cell Coordinates (integer cast)
game->ray.map_x = (int)player.pos_x;
game->ray.map_y = (int)player.pos_y;

// 4. Delta Distance (distance ray travels to cross 1 cell)
game->ray.delta_dist_x = fabs(1.0 / game->ray.ray_dir_x);
game->ray.delta_dist_y = fabs(1.0 / game->ray.ray_dir_y);
```

The Vector Camera Math Explained

- camera_x = -1.0 -> Left edge of screen
- camera_x =  0.0 -> Dead center of screen
- camera_x = +1.0 -> Right edge of screen

Vector Addition:
Ray_Dir = Dir + Plane * camera_x

Delta Distance Simplification:
Using similar triangles, the distance along the ray to cross 1 horizontal unit is exactly |1.0 / ray_dir_x|.
If ray_dir_x == 0, delta_dist_x = 1e30 (infinity), preventing division by zero!

# 10. RAYCASTING STEP 2: DDA STEP & SIDE DISTANCE

Code Location: src/raycast/raycast_main.c: calc_step_and_side_dist()

```
calc_step_and_side_dist(t_game *game)

// X-Direction Setup:
if (game->ray.ray_dir_x < 0) {
    game->ray.step_x = -1;
    game->ray.side_dist_x = (pos_x - map_x) * delta_dist_x;
}
else {
    game->ray.step_x = 1;
    game->ray.side_dist_x = (map_x + 1.0 - pos_x) * delta_dist_x;
}

// Y-Direction Setup (same logic):
if (game->ray.ray_dir_y < 0) {
    game->ray.step_y = -1;
    game->ray.side_dist_y = (pos_y - map_y) * delta_dist_y;
}
else {
    game->ray.step_y = 1;
    game->ray.side_dist_y = (map_y + 1.0 - pos_y) * delta_dist_y;
}
```

What is side_dist and step?

• step_x / step_y (+1 or -1):
Indicates which direction we step in the grid matrix (right/left or down/up).

• side_dist_x / side_dist_y:
Distance the ray must travel from player's starting point to reach the VERY FIRST grid boundary line!

• Geometric intuition:
If player is at x = 3.7:
- Moving right (+1): distance to x = 4.0 is (4.0 - 3.7) = 0.3 * delta_x.
- Moving left (-1): distance to x = 3.0 is (3.7 - 3.0) = 0.7 * delta_x.

# 11. RAYCASTING STEP 3: DDA GRID INTERSECTION

Code Location: src/raycast/raycast_main.c: perform_dda()

```
perform_dda(t_game *game)

while (game->ray.hit == 0)
{
   // Jump to next grid line in whichever axis is closer
   if (game->ray.side_dist_x < game->ray.side_dist_y)
   {
      game->ray.side_dist_x += game->ray.delta_dist_x;
      game->ray.map_x += game->ray.step_x;
      game->ray.side = 0; // East/West wall hit
   }
   else
   {
      game->ray.side_dist_y += game->ray.delta_dist_y;
      game->ray.map_y += game->ray.step_y;
      game->ray.side = 1; // North/South wall hit
   }
   // Did we strike a wall?
   if (game->map[game->ray.map_y][game->ray.map_x] == '1')
      game->ray.hit = 1;
}
```

Why DDA is Flawless

- Zero floating point errors: every step jumps by integer grid indices (+1 or -1).
- Impossible to "tunnel" through thin walls: every square intersected by the line of sight is checked in order.
- Minimal operations: just an addition and integer comparison.

side Variable:

- side = 0 means the ray hit a vertical boundary (X wall face).
- side = 1 means the ray hit a horizontal boundary (Y wall face).
- Crucial later to determine which texture (NO, SO, WE, EA) to draw!

# 12. RAYCASTING STEP 4: DISTANCE MATH & FISHEYE FIX

Code Location: src/raycast/raycast_main.c: calc_wall_projection()

```
calc_wall_projection(t_game *game)

// 1. Orthogonal Distance (eliminates fisheye!)
if (game->ray.side == 0)
    perp_wall_dist = game->ray.side_dist_x - game->ray.delta_dist_x;
else
    perp_wall_dist = game->ray.side_dist_y - game->ray.delta_dist_y;

// 2. Projected Column Height
line_height = (int)(SCREEN_HEIGHT / perp_wall_dist);

// 3. Screen Draw Coordinates (centered vertically)
draw_start = -line_height / 2 + SCREEN_HEIGHT / 2;
if (draw_start < 0) draw_start = 0;

draw_end = line_height / 2 + SCREEN_HEIGHT / 2;
if (draw_end >= SCREEN_HEIGHT) draw_end = SCREEN_HEIGHT - 1;
```

Why Straight Euclidean Distance Fails

• The Fisheye Effect:
If you use straight Euclidean hypotenuse sqrt(dx^2 + dy^2), rays hitting the wall edges travel farther than center rays. Flat walls curve into spherical fishbowls!

• Perpendicular Distance:
Measures orthogonal distance from camera plane to wall (effectively multiplying by cos(theta)). Flat walls remain perfectly flat and straight across the whole screen!

• Clamping:
draw_start and draw_end are clamped to [0, 599] so drawing never writes out of screen bounds.

# 13. RAYCASTING STEP 5: TEXTURE PIXEL MAPPING

Code Location: src/raycast/raycast_textures.c

1. Determine Texture Face & tex_x Coordinate

```
int get_tex_idx(t_game *game)
{
    if (game->ray.side == 0)
        return (game->ray.ray_dir_x > 0 ? 3 : 2); // EA or WE
    return (game->ray.ray_dir_y > 0 ? 1 : 0);     // SO or NO
}

// Exact point where wall was struck (wall_x in [0.0, 1.0))
if (side == 0)
    wall_x = player.pos_y + perp_dist * ray_dir_y;
else
    wall_x = player.pos_x + perp_dist * ray_dir_x;
wall_x -= floor(wall_x);

tex_x = (int)(wall_x * tex->width);
// Flip texture if viewing from opposite side so it's not mirrored!
```

2. Vertical Texture Scaling (draw_wall_column)

```
// Vertical step size along texture:
step = 1.0 * tex->height / line_height;

// Starting texture y position:
tex_pos = (draw_start - SCREEN_HEIGHT/2 + line_height/2) * step;

y = draw_start;
while (y <= draw_end)
{
    color = get_tex_color(tex, tex_x, (int)tex_pos);
    my_mlx_pixel_put(game, x, y, color);
    tex_pos += step;
    y++;
}
```

• Ensures texture scales seamlessly regardless of whether wall is 50px or 2000px tall!

# 14. CONTROLS, WALL SLIDING & ROTATION MATRIX

Code Location: src/utils/controls.c

move_player(): Independent Axis Wall Sliding

```
new_x = pos_x + move_x;
new_y = pos_y + move_y;
buf = 0.1;

// Check X movement with buffer:
if (map[py][(int)(new_x + buf)] != '1')
    player.pos_x = new_x;

// Check Y movement independently:
if (map[(int)(new_y + buf)][px] != '1')
    player.pos_y = new_y;
```

• Why this enables sliding: If walking into a vertical wall, X is blocked but Y continues moving smoothly along the wall!
• Buffer (0.1) keeps player safely away from wall seams.

rotate_player(): 2D Rotation Matrix

When pressing Left or Right arrow:

$$\begin{bmatrix} dir_x' \\ dir_y' \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} * \begin{bmatrix} dir_x \\ dir_y \end{bmatrix}$$

Code Implementation:

```
old_dir_x = dir_x;
dir_x = dir_x * cos(rot) - dir_y * sin(rot);
dir_y = old_dir_x * sin(rot) + dir_y * cos(rot);

old_plane_x = plane_x;
plane_x = plane_x * cos(rot) - plane_y * sin(rot);
plane_y = old_plane_x * sin(rot) + plane_y * cos(rot);
```

• Rotating BOTH vectors simultaneously preserves FOV!

# 15. BONUS: RADAR MINIMAP & 42 SEPARATION

Code Locations: src/bonus/minimap_bonus.c & Makefile

src/bonus/minimap_bonus.c: Radar HUD

- Top-left radar overlay with circular FOV.
- Radius: MM_RAD = 6 tiles around player.
- Tile Scale: MM_SCALE = 10 pixels per cell.
- Color Legend:
  - Wall ('1'): Dark Gray (0x333333)
  - Floor ('0'): Light Gray (0xCCCCCC)
  - Outside: Black (0x000000)
  - Player Indicator: Red 2x2 dot (0xFF0000) at center
- Loops dy = -6 to +6, dx = -6 to +6.
- The player stays centered; map moves relative to player!

Strict 42 Separation in Makefile

42 Rule: Mandatory executable MUST NOT execute bonus code!

Makefile Implementation:
- make -> compiles cub3D into obj/
  (No minimap_bonus.o, no -DBONUS flag)
- make bonus -> compiles cub3D_bonus into obj_bonus/
  (Includes minimap_bonus.o, activates -DBONUS=1)

In mlx_tools.c:

```
#ifdef BONUS
    draw_minimap(game);
#endif
```

- Completely independent build directories ensure zero interference between targets!

# 16. MEMORY LIFECYCLE & ZERO-LEAK AUDIT

Code Locations: src/utils/cleanup.c & src/utils/exit.c

Unified Cleanup: close_game()

```
int close_game(t_game *game)
{
    // 1. Free 2D map strings
    if (game->map) free_2d_array(game->map);
    // 2. Free 4 texture paths & XPM images
    free_textures(game);
    // 3. Destroy screen frame buffer image
    if (game->img) mlx_destroy_image(game->mlx, game->img);
    // 4. Destroy window
    if (game->win) mlx_destroy_window(game->mlx, game->win);
    // 5. Destroy X11 display connection
    destroy_mlx(game);
    exit(0);
}
```

• KEY_ESC and window [X] both call close_game().

Valgrind Memcheck Audit: 0 Leaks

Output on complex test maps:

```
==590== HEAP SUMMARY:
==590==   in use at exit: 0 bytes in 0 blocks
==590==   total heap usage: 171 allocs, 171 frees
==590==
==590== All heap blocks were freed -- no leaks are possible
==590== ERROR SUMMARY: 0 errors from 0 contexts
```

Defensive Error Handling:
All syntax errors (e.g. invalid RGB format) free temporary ft_split arrays before calling error_exit(). Zero leaks under any termination condition!

# 17. PEER EVALUATION DEFENSE CHEAT SHEET

Bulletproof answers to the 4 most common evaluator questions

Q1: What is DDA and why use it?
A: Digital Differential Analysis. Jumping grid boundary to boundary guarantees we never step through thin walls, avoiding slow float increments and trigonometry.

Q2: Why perpendicular distance instead of Euclidean?
A: Euclidean hypotenuse measures diagonal ray length, curving flat walls into fishbowls. Perpendicular distance measures orthogonal distance from the camera plane, keeping walls flat.

Q3: Why BFS flood fill instead of recursion?
A: Recursive flood fill exhausts stack frames on large open maps (causing stack overflow/crash). Iterative heap queue BFS is memory-safe and distinguishes outer spaces from illegal interior holes.

Q4: How do you prevent memory leaks when pressing ESC?
A: KEY_ESC is hooked directly to close_game(), which destroys all textures, MLX buffers, display connections, and frees the map 2D array.